Lista Teórica E3 — Gabarito

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Aula E3 — NLP + Classificação (Aplicação)

Exercício 1 — do texto à matriz. Considere três documentos:

d_1 = "o gato dorme", d_2 = "o cachorro dorme", d_3 = "o gato corre e o cachorro corre".

(a) Monte o vocabulário em ordem alfabética e escreva a matriz documento–termo de **contagens** ($3 \times V$).

(b) O `scikit-learn` calcula $\text{idf}(t) = \ln\left(\frac{1+n}{1+\text{df}(t)}\right) + 1$, onde $\text{df}(t)$ é o número de documentos que contêm t. Calcule o idf de *gato* e de *o*.

(c) Calcule o vetor TF-IDF de d_1, normalizado em norma ℓ_2. *Dados:* $\ln(4/3) \approx 0{,}2877$.

(d) Por que o termo *o* recebe o menor peso possível? E por que ainda assim ele não é zerado por essa fórmula?

Solução

(a) Vocabulário: *cachorro, corre, dorme, e, gato, o* — $V = 6$.

	cachorro	corre	dorme	e	gato	o
d_1	0	0	1	0	1	1
d_2	1	0	1	0	0	1
d_3	1	2	0	1	1	2

Repare no que se perdeu: a **ordem** das palavras. "o gato corre" e "corre o gato" dão exatamente a mesma linha. É o que o nome *bag of words* — saco de palavras — quer dizer.

(b) Com $n = 3$:

- *gato* aparece em d_1 e d_3, então $\text{df} = 2$ e $\text{idf} = \ln\left(\frac{4}{3}\right) + 1 \approx \mathbf{1{,}2877}$;
- *o* aparece nos três, $\text{df} = 3$ e $\text{idf} = \ln\left(\frac{4}{4}\right) + 1 = \ln 1 + 1 = \mathbf{1}$.

(c) O vetor bruto de d_1 é tf × idf, componente a componente. Só três entradas são não nulas:

$$\text{dorme} : 1 \times 1{,}2877, \qquad \text{gato} : 1 \times 1{,}2877, \qquad \text{o} : 1 \times 1.$$

A norma é $\sqrt{1{,}2877^2 + 1{,}2877^2 + 1^2} = \sqrt{1{,}6582 + 1{,}6582 + 1} = \sqrt{4{,}3164} \approx 2{,}0776$, e o vetor normalizado fica

$$\Big(0,\ 0,\ \underbrace{0{,}6198}_{\text{dorme}},\ 0,\ \underbrace{0{,}6198}_{\text{gato}},\ \underbrace{0{,}4813}_{\text{o}}\Big).$$

(d) Porque *o* aparece em **todos** os documentos, e um termo que aparece em todo lugar não distingue nada — é exatamente isso que o idf mede. Com $\text{df}(t) = n$, o argumento do logaritmo é $\frac{1+n}{1+n} = 1$ e o idf cai ao seu valor mínimo.
Mas ele não é *zero*: é 1, por causa do "+1" na fórmula. Sem esse termo, todo vocábulo presente em todos os documentos seria multiplicado por zero e sumiria da matriz. O "+1"

é uma decisão de projeto do `scikit-learn` (o parâmetro `smooth_idf` controla o outro "+1", o de dentro do logaritmo) para que nenhum termo seja descartado *pela fórmula* — descartar é papel do `min_df` e da lista de *stop words*, que são decisões explícitas de quem modela.

Exercício 2 — naive Bayes multinomial e a suavização de Laplace. Um filtro de spam foi treinado sobre um vocabulário de $V = 5$ palavras, com as contagens totais abaixo (soma sobre todas as mensagens de cada classe):

	grátis	clique	oferta	reunião	amanhã	total
spam	8	5	4	0	1	18
não-spam	1	0	1	6	7	15

As prioris estimadas são $\mathbb{P}(\text{spam}) = 0{,}4$ e $\mathbb{P}(\text{não-spam}) = 0{,}6$. O modelo multinomial estima

$$\mathbb{P}(w \mid c) \;=\; \frac{N_{wc} + \alpha}{N_c + \alpha V},$$

com $\alpha = 1$ (suavização de Laplace).

(a) Classifique a mensagem "clique oferta grátis", comparando as log-posterioris das duas classes. *Dados:* $\ln 0{,}4 = -0{,}9163$, $\ln 0{,}6 = -0{,}5108$.

(b) O que aconteceria com a log-posteriori de não-spam se $\alpha = 0$ (sem suavização)? Isso é razoável?

(c) Por que se trabalha com *logaritmos* das probabilidades, e não com as probabilidades diretamente?

(d) O modelo supõe que as palavras são independentes dentro de cada classe. Cite duas maneiras óbvias pelas quais isso é falso em texto. Por que o método funciona mesmo assim?

Solução

(a) Com $\alpha = 1$ e $V = 5$, o denominador é $18 + 5 = 23$ no spam e $15 + 5 = 20$ no não-spam.

	$\mathbb{P}(w \mid \text{spam})$	$\mathbb{P}(w \mid \text{não-spam})$
clique	$(5+1)/23 = 0{,}2609$	$(0+1)/20 = 0{,}0500$
oferta	$(4+1)/23 = 0{,}2174$	$(1+1)/20 = 0{,}1000$
grátis	$(8+1)/23 = 0{,}3913$	$(1+1)/20 = 0{,}1000$

Somando os logaritmos:

$$\ln \mathbb{P}(\text{spam} \mid \text{msg}) \propto -0{,}9163 - 1{,}3437 - 1{,}5261 - 0{,}9383 = \mathbf{-4{,}7244},$$

$$\ln \mathbb{P}(\text{não-spam} \mid \text{msg}) \propto -0{,}5108 - 2{,}9957 - 2{,}3026 - 2{,}3026 = \mathbf{-8{,}1117}.$$

Como $-4{,}7244 > -8{,}1117$, a mensagem é classificada como **spam**. (A diferença de 3,39 em log corresponde a uma razão de chances de $e^{3{,}39} \approx 30$ a favor de spam.)

(b) Sem suavização, $\mathbb{P}(\text{clique} \mid \text{não-spam}) = 0/15 = 0$, e o produto inteiro zera: $\mathbb{P}(\text{não-spam} \mid \text{msg}) = 0$ *exatamente*, ou $-\infty$ em log.

Não é razoável. Uma única palavra nunca vista naquela classe no treino **veta** a classe inteira, por mais que todas as outras palavras apontem para ela. E "nunca vista no treino" é banal em texto: o Exercício 3 mostra que a maioria dos termos aparece em pouquíssimos documentos. A suavização substitui a afirmação "é impossível" por "é raro", que é o que os dados de fato sustentam.

(c) Por **estabilidade numérica**. A posteriori é um produto de tantos fatores quanto palavras na mensagem, cada um bem menor que 1. Numa mensagem de 100 palavras com probabilidades típicas de 10^{-4}, o produto é da ordem de 10^{-400} — abaixo do menor *float* de precisão dupla representável ($\approx 10^{-308}$), e portanto arredondado para zero. As duas classes dariam zero, e a comparação seria impossível.
Em log, o produto vira soma e os valores ficam na casa das centenas negativas, perfeitamente representáveis. Como o logaritmo é estritamente crescente, comparar log-posterioris é equivalente a comparar posterioris.

(d) Duas maneiras óbvias:

- **colocações**: "São" e "Paulo" aparecem juntos muito mais do que o produto das frequências individuais previria; o mesmo para "cartão de crédito";
- **tema**: numa mensagem sobre futebol, ver "gol" aumenta muito a chance de ver "time" — as palavras de um mesmo campo semântico se atraem, dentro da própria classe.

Funciona mesmo assim: para **classificar** basta acertar qual log-posteriori é maior, e contar a mesma evidência duas vezes tende a deslocar as duas na mesma direção, preservando a ordem. Some-se a isso a economia de parâmetros — o multinomial estima V probabilidades por classe, e modelar dependências exigiria V^2 ou mais, o que é inviável com V na casa dos milhares.
O preço está nas **probabilidades**, como na Aula 07: elas saem empurradas para 0 e 1. Para decidir spam ou não, tanto faz; para dizer "esta mensagem tem 73% de chance de ser spam", não serve.

Exercício 3 — esparsidade, e o que fazer com ela. Numa coleção de 5 572 mensagens de SMS, o `CountVectorizer` produz um vocabulário de 8 672 termos, e a matriz resultante tem 73 916 entradas não nulas.

(a) Calcule a densidade da matriz. Se ela fosse guardada densamente, em *float* de 8 bytes, quanto ocuparia? E num formato esparso que guarda, para cada não-zero, o valor (8 bytes) e o índice da coluna (4 bytes)?

(b) Por que a matriz é tão esparsa? A mediana de palavras distintas por mensagem é 11; use isso para prever a densidade e compare com o valor do item (a).

(c) O que faz `CountVectorizer(min_df=5)`? Que efeito isso tem sobre o tamanho do vocabulário, e por que costuma *ajudar* o classificador?

(d) Cite um método do curso que **não** funcionaria bem sobre uma matriz esparsa desse tipo, e explique por quê.

Solução

(a) A densidade é $73\,916/(5\,572 \times 8\,672) = \mathbf{0{,}153}\%$ — menos de duas entradas em cada mil.

Densa: $5\,572 \times 8\,672 \times 8$ bytes $\approx 3{,}87 \times 10^8$ bytes $\approx$ **387 MB**. Esparsa: $73\,916 \times 12$ bytes $\approx$ **0,89 MB** (mais um vetor de ponteiros de linha, desprezível). Uma redução de **436 vezes**.
É o que torna o problema tratável num computador comum — e é por isso que todos os vetorizadores do `scikit-learn` devolvem matrizes `scipy.sparse`, não `numpy`.

(b) Porque cada mensagem usa uma fração ínfima do vocabulário. Com mediana de 11 palavras distintas, a linha típica tem 11 entradas não nulas em 8 672 colunas, o que prevê densidade $11/8\,672 \approx 0{,}127\%$ — próximo dos 0,153% medidos. (A diferença vem da cauda: algumas mensagens são bem mais longas que a mediana, e a densidade é governada pela *média*, não pela mediana.)
O vocabulário é a união do que *todas* as mensagens usam, e cada uma usa quase nada dele. Isso é uma propriedade estrutural de texto, não um acidente deste conjunto: o vocabulário cresce com o tamanho do corpus enquanto o comprimento de cada documento não.

(c) `min_df=5` descarta todo termo que apareça em menos de 5 documentos. O vocabulário encolhe muito — neste corpus, de 7 168 termos (no conjunto de treino) para 1 418 — um quinto. A razão é que 53,9% dos termos aparecem em *um único* documento: nomes próprios, erros de digitação, números.
Costuma ajudar por duas razões. Primeiro, **variância**: um termo que aparece em 2 documentos, ambos spam, recebe um coeficiente enorme e confiante baseado em duas observações — é ruído que o modelo trata como sinal. Segundo, **custo**: menos colunas, menos parâmetros, ajuste mais rápido.
O `min_df` é, portanto, um hiperparâmetro de **regularização** disfarçado de parâmetro de pré-processamento — e, como tal, pertence ao `Pipeline` e à grade de busca da Aula 06, não a uma decisão tomada uma vez no começo.

(d) O **KNN** (ou qualquer método baseado em distância euclidiana). Duas razões que se somam:

- é a **maldição da dimensionalidade** na sua forma mais extrema: com $p \approx 8\,700$, todos os pontos ficam praticamente equidistantes;
- pior, com esparsidade alta, duas mensagens quaisquer quase não compartilham palavras, então a distância entre elas é essencialmente $\sqrt{\|x_i\|^2 + \|x_j\|^2}$ — determinada pelo *comprimento* das mensagens, não pelo conteúdo delas.

Os métodos que funcionam bem aqui são os **lineares** — naive Bayes, logística, SVM linear — justamente porque um modelo linear em 8 700 dimensões esparsas tem, na prática, pouquíssimos parâmetros ativos por observação, e porque a penalização ℓ_2 ou ℓ_1 controla a variância sem precisar de nenhuma noção de vizinhança.

Exercício 4 — n-gramas e a explosão do vocabulário. O `CountVectorizer` aceita `ngram_range=(1,2)`, que acrescenta ao vocabulário todos os **pares de palavras consecutivas** além das palavras isoladas.

(a) Para os três documentos do Exercício 1, liste os bigramas de cada um. Quantos bigramas distintos existem ao todo?

(b) Que informação os bigramas recuperam, que o saco de palavras havia jogado fora? Dê um exemplo em que isso muda a classificação.

(c) Um corpus com V palavras distintas tem no máximo V^2 bigramas possíveis, mas na prática o número observado é muito menor. Por quê?

(d) Acrescentar bigramas aumenta muito o número de colunas. Isso agrava o problema do Exercício 3(d)? E o risco de superajuste?

Solução

(a)

- d_1: *o gato*, *gato dorme*;
- d_2: *o cachorro*, *cachorro dorme*;
- d_3: *o gato*, *gato corre*, *corre e*, *e o*, *o cachorro*, *cachorro corre*.

Distintos: *o gato*, *gato dorme*, *o cachorro*, *cachorro dorme*, *gato corre*, *corre e*, *e o*, *cachorro corre* — **8 bigramas**. Com `ngram_range=(1,2)` o vocabulário passaria de 6 para $6 + 8 = \mathbf{14}$ colunas, mais que o dobro, num corpus de três frases.

(b) Os bigramas recuperam, parcialmente, a **ordem** das palavras — a informação que o saco de palavras descartou.
O exemplo canônico em classificação de texto é a **negação**. As frases "não é bom" e "é bom" têm sacos de palavras quase idênticos (a segunda é um subconjunto da primeira), e um modelo unigrama vê "bom" nas duas e tende a classificar as duas como positivas. Com bigramas, "não é" e "é bom" são colunas distintas, e o modelo pode aprender que "não é" carrega peso negativo.
Em detecção de spam, o mesmo vale para expressões: "clique aqui" é muito mais informativo que "clique" e "aqui" separados.

(c) Porque a linguagem é fortemente restrita: a esmagadora maioria dos pares de palavras **nunca ocorre**. Sintaxe ("o" não é seguido de verbo), semântica ("verde" não modifica "segunda-feira") e simples convenção de uso eliminam quase todas as V^2 combinações.
Empiricamente, o número de bigramas distintos num corpus cresce aproximadamente com o número de *tokens*, e não com V^2: é a lei de Heaps, e a razão de fundo é a lei de Zipf — a distribuição de frequências das palavras tem cauda muito pesada, e a maioria delas aparece pouquíssimas vezes, logo tem pouquíssimos vizinhos possíveis observados.

(d) Para o KNN, agrava. Mais colunas com a mesma quantidade de informação por documento significa esparsidade ainda maior e distâncias ainda menos informativas.
Para o superajuste, o quadro é mais sutil, e a resposta é "depende do que se faz com as colunas". Sim, o número de parâmetros cresce, e um bigrama que aparece em duas mensagens de spam vira um preditor perfeito no treino e ruído fora dele — exatamente o mecanismo do Exercício 3(c). Mas:

- `min_df` corta justamente esses bigramas raríssimos, que são a maioria esmagadora dos novos;
- a penalização (ℓ_2 da logística, ou o C da SVM) controla os coeficientes das colunas que sobram;
- modelos lineares em alta dimensão esparsa toleram bem colunas extras inúteis, desde que regularizados — é a observação de Greenshtein e Ritov citada na Aula 02, em que p aparece só dentro de um logaritmo.

Na prática, `ngram_range` e `min_df` são escolhidos *juntos*, por validação cruzada, dentro do mesmo `Pipeline` — e é comum que a combinação vencedora inclua bigramas com um `min_df` mais alto do que o usado só com unigramas.